

Digital Electronics Circuit Designer: Project Report

C++17 · NAND/NOR-only circuit builder and simulator · Meeting 2 · 3 October 2026

1. Problem and objectives

The project shows that complex digital hardware can be built from just two primitive gates, **NAND** and **NOR**, and then simulates that hardware. Both gates are functionally complete: any Boolean function can be built from either one alone.

Objectives:

- Build NOT, AND, OR, XOR and XNOR from the two primitives.
- Compose them into a half adder, full adder, 4-bit ripple-carry adder, D latch, master/slave D flip-flop and a 4-bit synchronous counter.
- Simulate combinational and sequential circuits, including feedback and a clock.
- Present readings as a console table and as CSV files.
- Stretch goal: run the simulation on several threads with results identical to the single-threaded run.

2. Approach in one page

1. **One uniform building block.** Only `NandGate` and `NorGate` compute anything. Every other part is wiring.
2. **Composition.** A `Circuit` owns its sub-components and internal wires. A counter contains flip-flops, which contain latches, which contain NANDs. `collectGates()` flattens any circuit into its primitive gates, so the simulator never needs to know what an adder is.
3. **Two-phase, unit-delay simulation.** In each round every gate first *evaluates* its next output from the current wires, then every gate *commits* by writing its output wire. Rounds repeat until no wire changes (`settle()`).
4. **Why two phases.** Results do not depend on gate order, memory elements settle naturally, and because phase 1 only reads and phase 2 only writes, gates can run on different threads without locks.

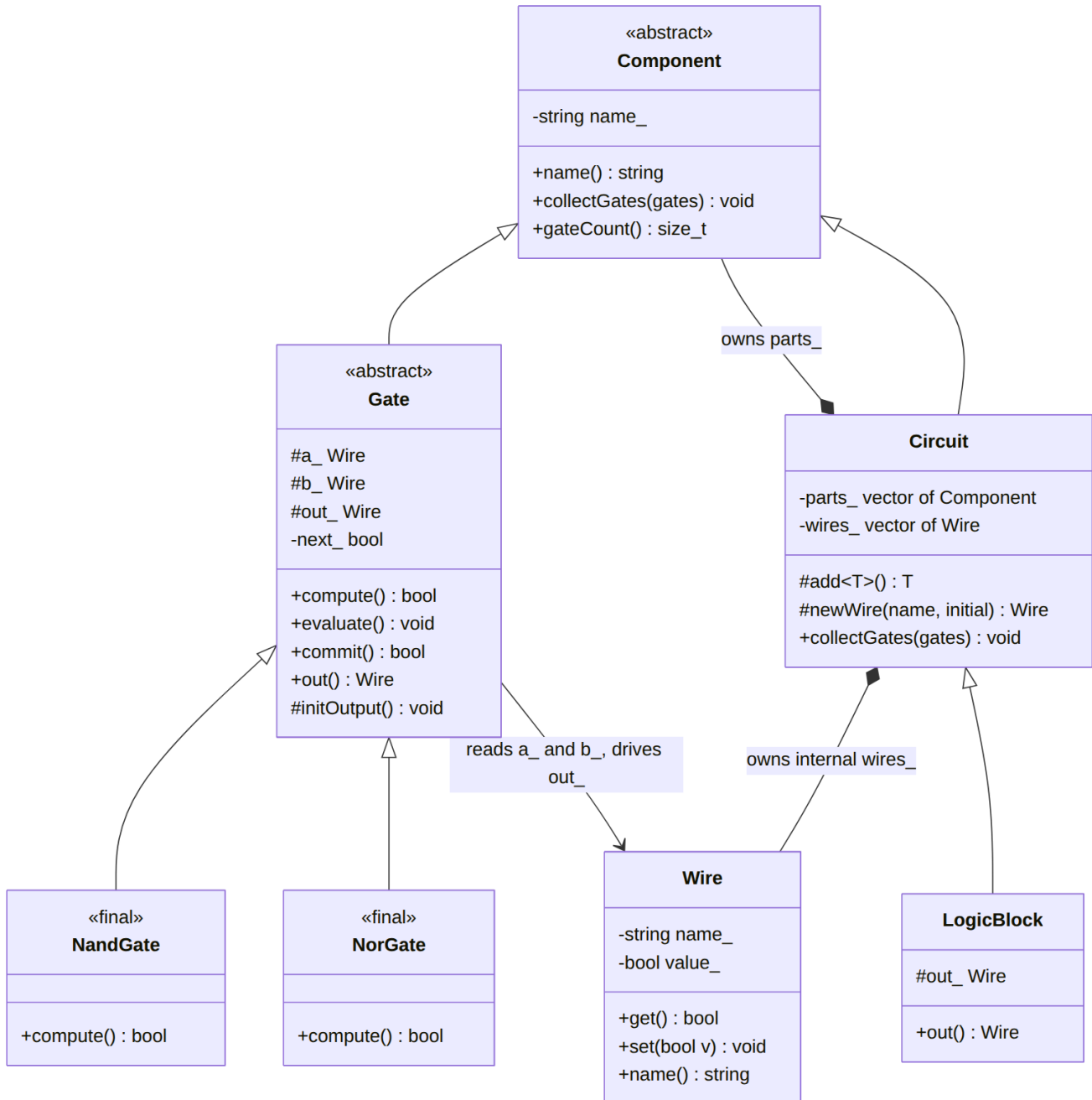
3. Architecture (layers)

Layer	Files	Role
Foundation	<code>Wire.h</code> , <code>Component.h</code> , <code>Gates.h</code>	Wire, Component, Gate, Circuit, NAND, NOR, derived gates
Circuits	<code>Circuits.h</code>	Adders, latch, flip-flop, buffer, counter
Engine	<code>Simulator.h/.cpp</code> , <code>SimulationResult.h</code>	Settle loop (sequential and threaded), probes, clock, result table
Output	<code>Output.h/.cpp</code> , <code>main.cpp</code>	Console and CSV output, three demos
Verification	<code>tests.cpp</code>	Automatic checks, exit code 0 means all passed

4. UML class diagrams

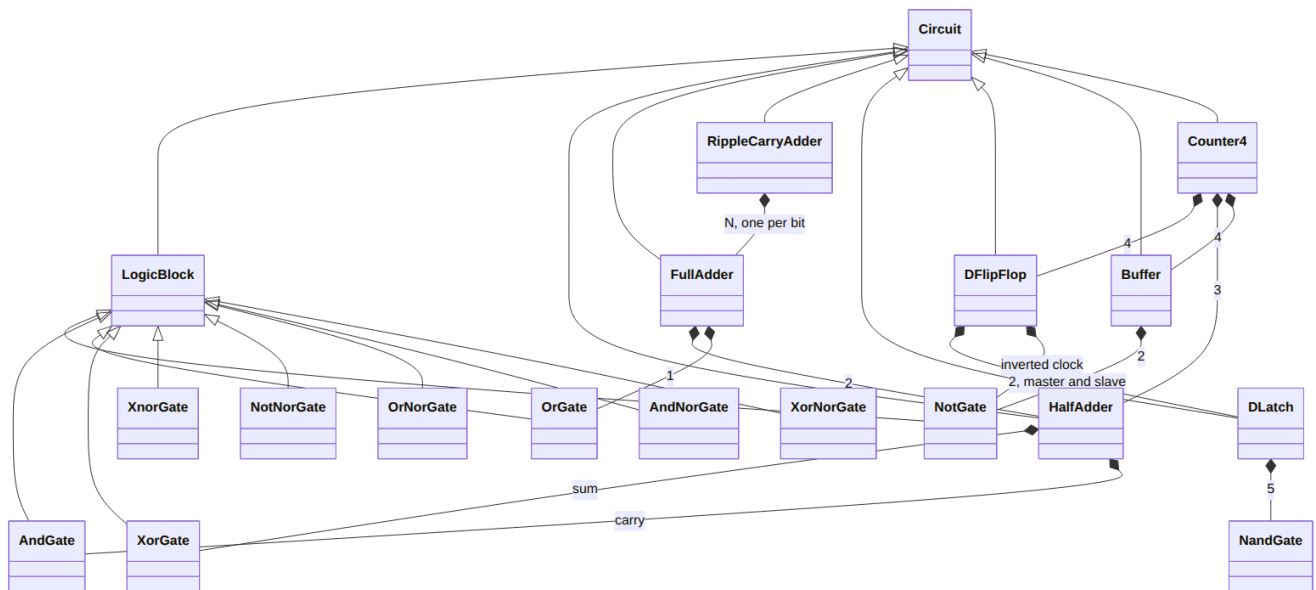
4.1 Core hierarchy

`Component` is the abstract root. A `Gate` is a primitive with two inputs and one output wire. A `Circuit` is a component built from other components. Pointers to wires are held, not owned, by gates.

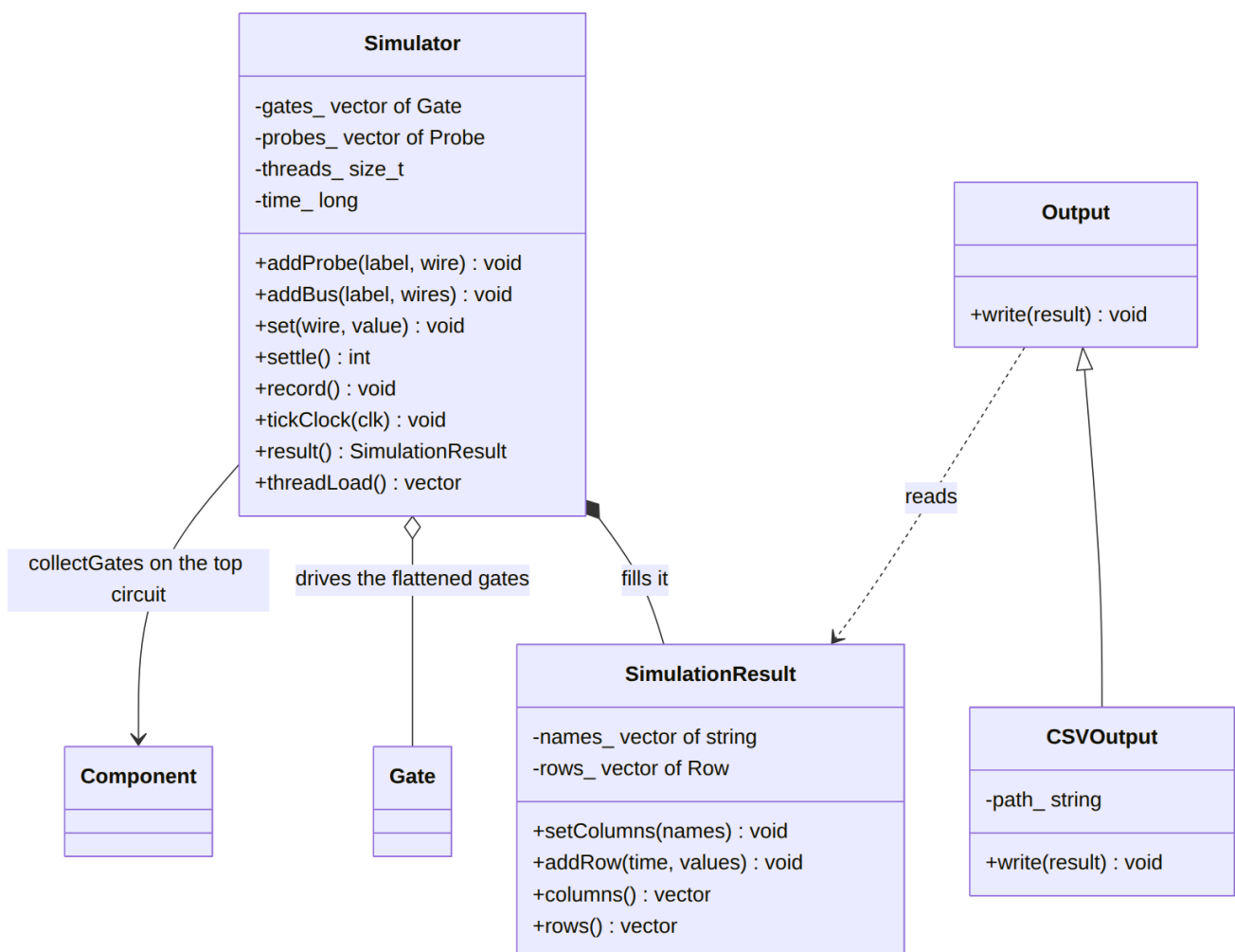


4.2 Derived gates and circuits

All derived gates and circuits inherit from `Circuit` and do their wiring inside their constructors. Solid diamonds mean the circuit owns instances of the part.

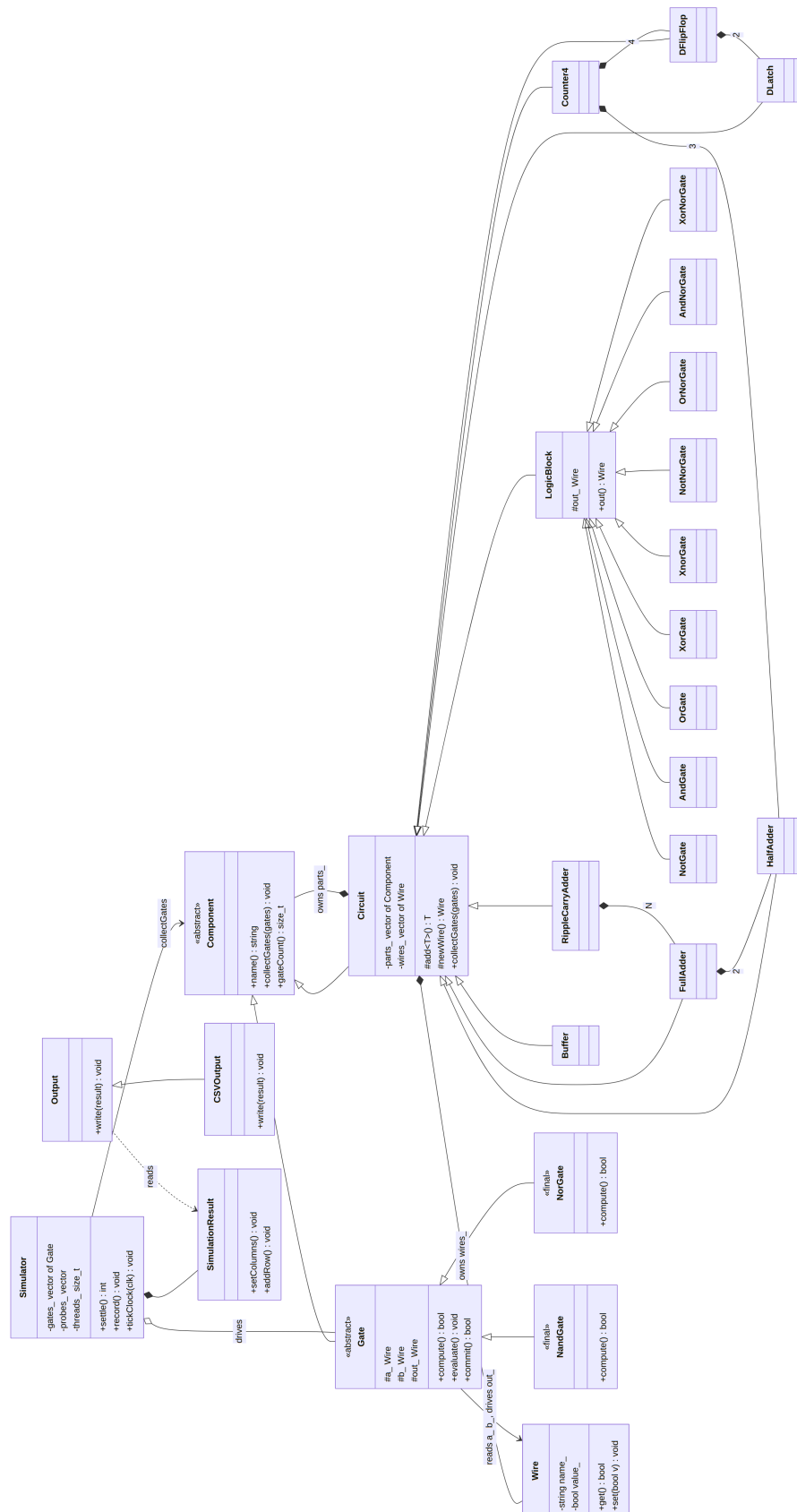


4.3 Simulation engine and output



4.4 Complete class diagram

All classes in one view. Hollow triangles show inheritance, filled diamonds show ownership, and the open diamond shows the simulator using gates it does not own. The diagram is turned sideways to fit the page; a landscape copy is supplied as a separate file.



5. OOP workflow and concepts used

The workflow in one line: **build the circuit by constructing objects, flatten it, simulate it, read the result.**

OOP concept	Where it appears
Abstraction	<code>Component</code> and <code>Gate</code> are abstract. <code>compute()</code> and <code>collectGates()</code> are pure virtual, so callers depend only on the interface.
Encapsulation	<code>Wire</code> keeps its value private. <code>Gate</code> hides its input and output wires and exposes only <code>evaluate()</code> , <code>commit()</code> and <code>out()</code> .
Inheritance	<code>Component</code> → <code>Gate</code> → <code>NandGate</code> / <code>NorGate</code> . <code>Component</code> → <code>Circuit</code> → <code>LogicBlock</code> → <code>NotGate</code> , <code>AndGate</code> and the other gates. <code>Circuit</code> → adders, latch, flip-flop, counter.
Polymorphism	<code>Simulator</code> holds <code>Gate</code> pointers and calls <code>evaluate()</code> and <code>commit()</code> without knowing whether a gate is NAND or NOR. <code>Output::write</code> is overridden by <code>CSVOutput</code> .
Composition	A <code>Circuit</code> owns its parts through <code>unique_ptr</code> . A full adder is two half adders and an OR. A counter is four flip-flops and an incrementer.
Templates	<code>Circuit::add<T>()</code> constructs any sub-component and returns a typed pointer. The tests use a templated <code>gateTest<G></code> .
Ownership and RAI	<code>unique_ptr</code> for parts and wires, and copying is deleted for <code>Wire</code> and <code>Component</code> so a wire is never duplicated by accident.
Constructor-based wiring	Each circuit connects its internals inside its constructor, so a finished object is always fully wired.

5.1 Where inheritance and polymorphism are used, and why

Inheritance

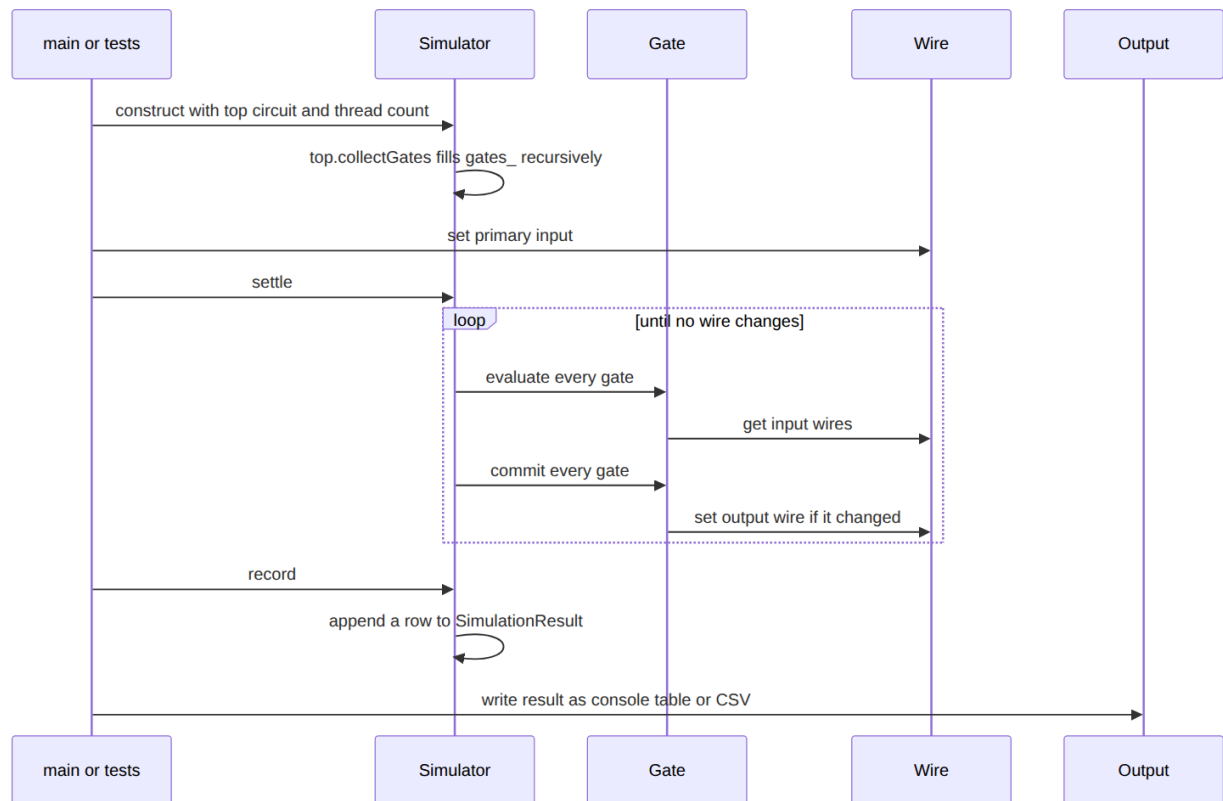
Hierarchy	Why it is used
<code>Component</code> → <code>Gate</code> and <code>Circuit</code>	A gate and a whole circuit share one type, so a <code>Circuit</code> can hold any mix of gates and sub-circuits in one list of <code>Component</code> pointers.
<code>Gate</code> → <code>NandGate</code> , <code>NorGate</code>	<code>Wires</code> , <code>evaluate()</code> and <code>commit()</code> are written once in <code>Gate</code> . The two primitives differ only in their truth table.
<code>Circuit</code> → <code>LogicBlock</code> → <code>NotGate</code> , <code>AndGate</code> , <code>XorGate</code> and the other gates	<code>LogicBlock</code> adds the shared single-output accessor. Each gate only wires its internals.
<code>Circuit</code> → adders, latch, flip-flop, buffer, counter	Each reuses the circuit's ownership of parts and wires. A new circuit is one new class with no change to existing code.
<code>Output</code> → <code>CSVOutput</code>	Reuses the output interface but writes to a file instead of the console.

Polymorphism

Virtual function	How it works	Why it matters
<code>Component::collectGates()</code> (pure virtual)	A <code>Gate</code> adds itself to the list. A <code>Circuit</code> calls <code>collectGates()</code> on each of its parts.	The simulator calls one function on the top circuit and gets every primitive gate, without checking what kind of circuit it is.
<code>Gate::compute()</code> (pure virtual)	<code>NandGate</code> and <code>NorGate</code> each override it. <code>Gate::evaluate()</code> calls it.	The simulator holds <code>Gate</code> pointers and runs a mixed list of NAND and NOR gates in one loop.
<code>Component</code> virtual destructor	Circuits own parts through <code>unique_ptr<Component></code> .	Deleting through the base pointer destroys the full derived object correctly.
<code>Output::write()</code> (virtual)	<code>CSVOutput</code> overrides it.	Shows overriding. In <code>main.cpp</code> the calls are made on the concrete objects, not through a base pointer.

`evaluate()` and `commit()` are not virtual themselves. The polymorphism happens inside `evaluate()`, which calls the virtual `compute()`.

Workflow of one simulation



In multi-threaded mode the gates are split into contiguous ranges, one per thread. Each round has a barrier after the evaluate phase and another after the commit phase. An atomic counter totals the changes, and a round limit of 5000 stops oscillating circuits with an error.

6. Circuit constructions and gate counts

Circuit	Construction	Primitive gates
NOT	NAND(a, a)	1
AND	NAND followed by NOT	2
OR	NAND(NOT a, NOT b)	3
XOR	classic 4-NAND form	4
XNOR	NOT of XOR	5
AND, OR, XOR from NOR	dual constructions	3, 2, 5
Half adder	XOR for sum, AND for carry	6
Full adder	2 half adders plus OR	15
4-bit ripple-carry adder	4 full adders, carry chained	60
D latch	NOT plus 4 NANDs, cross-coupled pair holds state	5

Circuit	Construction	Primitive gates
D flip-flop	master latch on inverted clock, slave latch on clock	11
4-bit counter	4 flip-flops, a NOT and 3 half adders as incrementer, 4 buffers	71

The `Buffer` (two inverters) lets the incrementer drive the flip-flop D wires, which are created before the incrementer is built.

7. Testing and results

`make test` runs 48 checks, all passing:

- Truth tables of NAND, NOR and every derived gate.
- All 512 input combinations (two 4-bit operands and carry-in) of the ripple-carry adder, run with 1 thread and with 4 threads.
- D latch: power-on value, transparent when enabled, holds when disabled.
- Counter: 40 clock cycles with wrap-around from 15 to 0, and the 4-thread result is identical to the 1-thread result.

The demo (`make run`) prints the gate truth table, runs all 512 adder vectors with 0 wrong, runs 20 counter clock cycles, and writes `gates.csv`, `adder.csv` and `counter.csv`.

Thread scaling note: on a single-core test machine the demo ran in 0.005 s with 1 thread, 0.13 s with 4 and 0.32 s with 8. With only 25 to 71 gates per circuit, thread start-up and barrier cost exceed the work. The threaded mode is correct and lock-free, but it is a design demonstration, not a speed-up at this circuit size.

8. Team contributions

Each member should be able to explain their files, one design decision and one demo.

Person	Files	Contribution	Be ready to explain
1: [name]	Wire.h, Gates.h, Circuits.h	The Wire class. NandGate and NorGate, and every derived gate built from them (NOT, AND, OR, XOR, XNOR and the NOR versions). The half adder, full adder, 4-bit ripple-carry adder, D latch, D flip-flop, buffer and 4-bit counter.	How XOR is built from 4 NANDs. How the cross-coupled NAND latch remembers. Why the flip-flop is master/slave. How the counter increments and why Buffer is needed.
2: [name]	Component.h, SimulationResult.h, Simulator.h, Simulator.cpp	The Component, Gate and Circuit base classes, including evaluate(), commit(), collectGates() and add<T>(). The result table. The simulator: settle loop, multi-threaded mode with barriers, probes, buses and the clock tick.	Why Gate has separate evaluate() and commit(). Why two-phase evaluation is lock-free. How the barrier and the 5000-round limit work. Why Circuit uses add<T>().
3: [name]	Output.h, Output.cpp, main.cpp, tests.cpp	The Output base class and CSVOutput. The three demos (gate truth table, 512 adder vectors, counter run). The test suite of 48 checks.	How Output is overridden by CSVOutput. What each demo proves. How the tests show threaded results equal sequential results.

9. Limitations

- Unit-delay timing: every gate takes one round, with no realistic propagation delays or glitch analysis.
- The counter has no reset or enable input.
- Threading brings no speed-up at this circuit size, and threads are re-created on every settle() call.
- Circuits are built in C++ code, not loaded from a description file.

10. Possible extensions

Extension	What it adds	Effort
Counter reset and enable	Synchronous reset, count enable, up/down mode	Small

Extension	What it adds	Effort
More building blocks	Multiplexer, decoder, comparator, subtractor, SR, JK and T flip-flops	Small to medium
Shift register and register file	Serial-in parallel-out register built from flip-flops	Medium
4-bit ALU slice	Add, subtract, AND, OR, XOR selected by a control input	Medium
Waveform output	Export a VCD file to view signals in GTKWave	Medium
Persistent thread pool	Fixes thread start-up cost, then benchmark on a 32-bit adder or large arrays	Medium
Per-gate delays, event-driven engine	Realistic timing and glitches, only active gates evaluated	Larger
Netlist file loader	Describe circuits in a text file and load them at run time	Larger
Finite-state machine or simple CPU	Traffic light controller, then a tiny datapath	Larger

11. Build and run

```

make          # builds circuit_designer and tests
make test     # 48 automatic checks
make run      # demo with 4 threads, writes gates.csv, adder.csv, counter.csv
./circuit_designer 1  # single-threaded
./circuit_designer 8  # 8 threads

```

Without make: `g++ -std=c++17 -O2 -pthread main.cpp Simulator.cpp Output.cpp -o circuit_designer.`